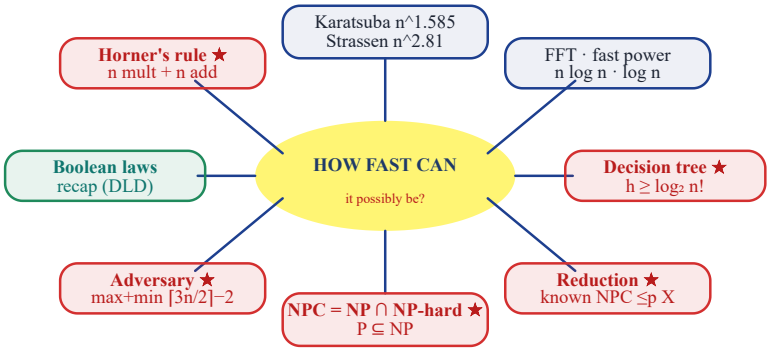
Algebraic Simplification, Lower Bound & NP-Completeness

Horner · fast power · Karatsuba · Strassen · FFT · decision trees · adversaries · P, NP, NP-hard, NP-complete: exam-ready in eleven sheets

Eleven sheets: how rewriting an algebraic problem makes it cheaper, a compact Boolean-law recap, what a **lower bound** really is and the three ways to prove one, the table of proven bounds, and then the complexity classes, reductions and what to do when a problem is NP-complete.



↑ the whole lesson: red = high-yield, green = recap from another chapter

Syllabus coverage

The syllabus line for Data Structures and Algorithms ends: "**Algebraic simplification and transformations; Lower bound theory; NP-completeness, NP-hard and NP-complete problems.**" All three clauses are covered here. In an algorithms syllabus "algebraic simplification" means **algebraic algorithms** — rewriting polynomials, powers and products so they cost fewer operations — so that is the main thread; Boolean simplification gets a one-sheet recap, as K-maps belong to the Digital Logic chapter. **Approximation algorithms** are in the flow / approximation / parallel lesson of this chapter.

High-yield list

① Horner: n mult + n add
② Karatsuba n^1.585 · Strassen n^2.81
③ Fast power: O(log n) multiplications
④ Ω = lower bound, not "best case"
⑤ **Comparison sort Ω(n log n) ★**
⑥ Max: n−1 · max+min: ⌈3n/2⌉−2
⑦ 2nd largest: n + ⌈log₂ n⌉ − 2
⑧ NP = verifiable in poly time
⑨ NPC = NP ∩ NP-hard ★
⑩ Cook–Levin 1971: SAT first NPC
⑪ Prove X hard: known NPC ≤p X ★
⑫ 2-SAT ∈ P, 3-SAT NP-complete

Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

Contents

✓ Transform-and-conquer, Horner … p. 2
✓ Fast power, Karatsuba, Strassen, FFT … p. 3
✓ Boolean recap · O, Ω, Θ … p. 4
✓ Lower bounds & the decision tree … p. 5
✓ P, NP, co-NP, NP-hard, NPC … p. 7
✓ Reductions & Cook–Levin … p. 8
✓ Easy/hard twins, coping … p. 9
✓ Night-before sheet … p. 10

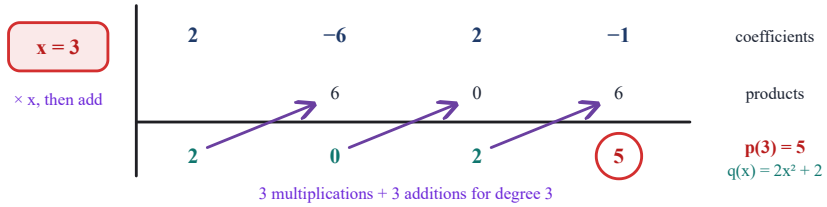
✓ Adversaries & the bounds table … p. 6 ✓ Practice MCQs + written … p. 11

1 Transform-and-Conquer and Horner's Rule

Definition: **Algebraic simplification / transformation** means **rewriting a problem or its formula into an equivalent form that is cheaper to compute**: same answer, fewer operations. The strategy is called **transform-and-conquer**; it has three flavours.

Flavour	Idea	Classic examples
Instance simplification	simpler instance of the **same** problem	**presorting**: element uniqueness $O(n^2)$ → sort + scan $O(n \log n)$; Gaussian elimination
Representation change	same instance, **new representation**	**Horner's form**; binary exponentiation; point-value form; heaps, AVL trees
Problem reduction	a **different** problem with a known algorithm	`lcm(m,n) = m·n / gcd(m,n)`; `max f = -min(-f)`; k-step paths = `A^k`

Horner's rule: Factor x out again and again: $p(x) = a_n x^n + \dots + a_1 x + a_0 = (\dots((a_n x + a_{n-1})x + a_{n-2})x + \dots)x + a_0$. One pass from the leading coefficient: **n multiplications + n additions** for degree n, with no powers computed at all.



Horner = synthetic division: $p(x) = 2x^3 - 6x^2 + 2x - 1$ at $x = 3$ (check: $54 - 54 + 6 - 1 = 5$)

```
HORNER(a[0..n], x):
  p = a[n]
  for i = n - 1 downto 0:
      p = p · x + a[i]      // 1 mult, 1 add
  return p                  // Θ(n)
```

Method (degree n)	Mult.	Add.
each term $a_i x^i$ from scratch	$n(n+1)/2$	n
keep a running power x^i	$2n - 1$	n
Horner	**n**	n

Exam trap ✍ three: **1.** Degree n has **n + 1** coefficients, but Horner does **n** multiplications, not n + 1 or 2n. **2.** Horner is proved **optimal**: no general method uses fewer than n multiplications. **3.** The last value is **$p(x_0)$** (remainder theorem); the rest are the quotient.

Evaluation ↔ interpolation: A degree-n polynomial is fixed by its **n + 1 coefficients** or by its values at **n + 1 distinct points**. **Evaluation** at n + 1 points by Horner: $O(n^2)$. **Interpolation** (Lagrange / Newton): **$O(n^2)$**. Products are **O(n)** in point-value form — the FFT idea.

Memory trick: "**Horner = nested brackets = n and n.**" Binary 1101 → `((1·2 + 1)·2 + 0)·2 + 1 = 13` is Horner with x = 2.

2 Fast Power, Karatsuba, Strassen and the FFT

Fast (binary) exponentiation: `x^n = (x^(n/2))²` (n even) or `x·(x^((n−1)/2))²` (n odd): one squaring per bit, at most **2⌊log₂ n⌋ multiplications** instead of n − 1 ⇒ **O(log n)**. x^{13} ($13 = 1101_2$): $x \to x^2 \to x^3 \to x^6 \to x^{12} \to x^{13}$ = **5** multiplications, not 12.

Worked: modular power ✍️: 3^{13} mod 7: $3^1 = 3$, $3^2 = 9 \equiv 2$, $3^4 \equiv 4$, $3^8 \equiv 16 \equiv 2$. $13 = 8 + 4 + 1 \Rightarrow 3^{13} \equiv 2 \cdot 4 \cdot 3 = 24 \equiv$ **3** (mod 7). Reducing after every step keeps numbers small — this is how **RSA** and Diffie–Hellman compute huge powers.

Karatsuba multiplication (1960): Split two n-digit numbers: `x = a·10^m + b`, `y = c·10^m + d`. Schoolbook needs four products ac, ad, bc, bd. The algebraic trick **`ad + bc = (a + b)(c + d) − ac − bd`** needs only **three**. `T(n) = 3T(n/2) + O(n)` ⇒ **Θ(n^log₂3) ≈ Θ(n^1.585)**: the recursion tree has 3^k calls at depth k, so the n^log₂3 leaves dominate.

Worked example ✍️ 1234 × 5678: a = 12, b = 34, c = 56, d = 78 (m = 2). ac = 672, bd = 2652, (a + b)(c + d) = 46 × 134 = 6164 ⇒ ad + bc = 6164 − 672 − 2652 = 2840. Result = $672 \cdot 10^4 + 2840 \cdot 10^2 + 2652$ = **7,006,652** ✓ with three products.

Strassen (1969): Split n × n matrices into four n/2 blocks. Block multiplication needs 8 products; Strassen's identities need **7 products** (plus 18 block additions). `T(n) = 7T(n/2) + Θ(n²)` ⇒ **Θ(n^log₂7) ≈ Θ(n^2.81)** vs Θ(n^3). Worth it only for large n; less numerically stable.

FFT polynomial multiplication: Coefficient form: **Θ(n^2)**. Instead **evaluate** both at 2n complex roots of unity (FFT), **multiply pointwise** (O(n)), **interpolate** back (inverse FFT) ⇒ **O(n log n)**. FFT: `T(n) = 2T(n/2) + O(n)` (Cooley–Tukey, 1965).

Problem	Classical	Method	Recurrence	Result
evaluate degree-n poly	Θ(n^2)	Horner	one loop	Θ(n)
power x^n	n − 1	squaring	T(n/2) + O(1)	O(log n)
n-digit × n-digit	Θ(n^2)	Karatsuba	3T(n/2) + O(n)	**Θ(n^1.585)**
n × n matrices	Θ(n^3)	Strassen	7T(n/2) + O(n^2)	Θ(n^2.81)
poly × poly (degree n)	Θ(n^2)	FFT	2T(n/2) + O(n)	Θ(n log n)

Exam trap ✍️: Strassen's exponent is **log₂ 7 ≈ 2.807**, not 2.376 (that is the impractical Coppersmith–Winograd line). With 4 and 8 half-size products the exponent stays 2 and 3 — the saving is the **one** product dropped. Matrix product's trivial lower bound is **Ω(n^2)**.

Memory trick: "**Save one multiplication, lower the exponent.**" a half-size subproblems with cheap combining ⇒ **n^log₂ a** (master theorem): 4 → 3 gives log₂ 3 ≈ 1.585; 8 → 7 gives log₂ 7 ≈ 2.807.

3 Boolean Simplification: a Compact Recap

Law	OR form	AND form (dual)
Identity	A + 0 = A	A · 1 = A
Null (dominance)	A + 1 = 1	A · 0 = 0
Idempotent	A + A = A	A · A = A
Complement	A + A′ = 1	A · A′ = 0
Distributive	A(B + C) = AB + AC	**A + BC = (A + B)(A + C)**
Absorption	A + AB = A	A(A + B) = A
De Morgan	(A + B)′ = A′B′	(AB)′ = A′ + B′
Consensus	AB + A′C + BC = AB + A′C	(A+B)(A′+C)(B+C) = (A+B)(A′+C)

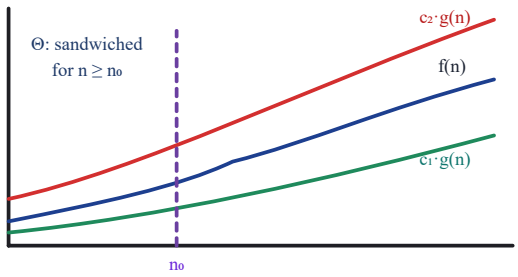
Worked ✍ two classics: **F = AB + AB′** = A(B + B′) [distributive] = A·1 [complement] = **A** [identity].
G = A + A′B = (A + A′)(A + B) [OR-distributive] = 1·(A + B) = **A + B**.
Consensus proof: BC = BC(A + A′) = ABC + A′BC, absorbed by AB and A′C.

The IT link: compilers: Optimising compilers do **algebraic simplification** on code: `x*1 → x`, `x+0 → x`, constant folding `2*3.14 → 6.28`, and **strength reduction** `x*2 → x<<1`, `x*x` instead of `pow(x,2)`. Minterms, K-maps and Quine–McCluskey: Digital Logic chapter.

Exam trap: **A + 1 = 1**, not A. **A + BC = (A + B)(A + C)** holds in Boolean algebra but not in ordinary algebra. **Duality:** swap + ↔ · and 0 ↔ 1 (variables untouched) and a law stays true.

4 Asymptotic Bounds: O, Ω, Θ Stated Correctly

Definitions: **f = O(g)**: ∃ c > 0, n_0 with $f(n) \le c \cdot g(n)$ for all $n \ge n_0$ (upper bound).
f = Ω(g): ∃ c > 0, n_0 with $f(n) \ge c \cdot g(n)$ for all $n \ge n_0$ (**lower bound**).
f = Θ(g): both — $c_1 g(n) \le f(n) \le c_2 g(n)$ (tight).
Strict versions: o and ω.



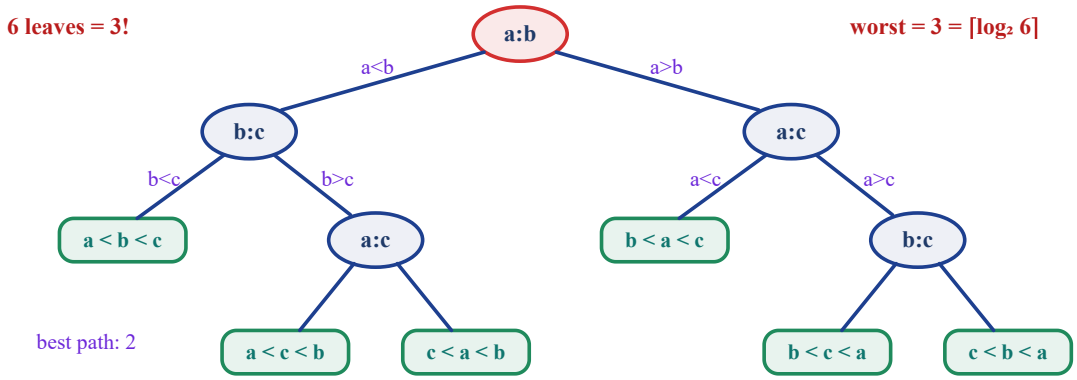
f = Θ(g) beyond n_0

Common misconception ✍: "Ω means best case, O means worst case" is **wrong**. O, Ω and Θ bound **functions**; the best, average and worst case are three different functions, and each can be given an O, Ω or Θ. Insertion sort: worst case **Θ(n²)** (so it is also Ω(n²)), best case **Θ(n)**. A **lower bound of a problem** is an Ω on the **worst case of every algorithm** — next sheet.

5 Lower Bound Theory and the Decision-Tree Argument

Definition: A **lower bound** L(n) of a problem is a cost that **every algorithm must pay in the worst case**, in a stated model (e.g. "comparisons only"). It is a statement about the **problem**, not one algorithm. The **upper bound** is the cost of the best known algorithm. When they meet, that algorithm is **optimal** (merge sort, heapsort: Θ(n log n) comparisons). A gap is an open problem: matrix product has Ω(n²) vs O(n^2.37…).

Technique	Idea	Classic result
Trivial (I/O size)	read every input, write every output	matrix product Ω(n²); all permutations Ω(n!)
Decision tree	k answers need ≥ $\lceil \log_2 k \rceil$ yes/no tests	sorting **Ω(n log n)**; search $\lceil \log_2(n+1) \rceil$
Adversary	answers each test to force most work	max n − 1; max + min ⌈3n/2⌉ − 2
Problem reduction	hard P reduces cheaply to Q ⇒ Q as hard	sorting ≤ convex hull ⇒ Ω(n log n)



a correct decision tree for sorting a, b, c: every internal node is one comparison, every leaf one ordering

The proof ✍ 4 marks: Any comparison sort on n distinct keys is a **binary decision tree**. It must reach a different leaf for each of the **n!** orderings ⇒ ≥ n! leaves. A binary tree of height h has ≤ 2^h leaves ⇒ $2^h \geq n!$ ⇒ **$h \geq \log_2(n!)$**. Since $n! \geq (n/2)^{(n/2)}$, $\log_2 n! \geq (n/2) \cdot \log_2(n/2) = \Omega(n \log n)$. (Stirling: $\log_2 n! \approx n \log_2 n - 1.44n$.)

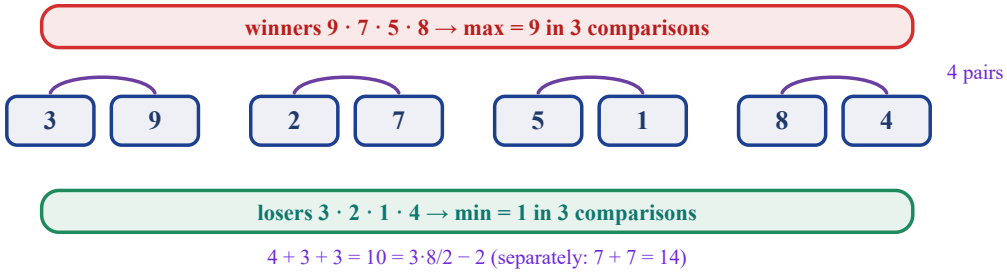
n	n!	$\lceil \log_2 n! \rceil$ = min. worst-case comparisons
3	6	**3**
4	24	5
5	120	7 (achieved by merge-insertion)
10	3,628,800	22

Exam trap ✍: The bound covers **comparison** sorts only. **Counting sort O(n + k)**, **radix sort O(d(n + k))** and bucket sort (O(n) average) use key values as array indices, so they are not limited by it. It also holds for the **average** case: the average leaf depth is ≥ $\log_2 n!$.

6 Adversary Arguments and Proven Lower Bounds

Adversary argument: Picture the algorithm asking questions ("is $a_i < a_j$?") and a malicious **adversary** answering them — always **consistently** with some input, but choosing the answer that tells the algorithm least. If the adversary can keep two different answers possible for k − 1 questions, every algorithm needs **k**. **Max of n:** each of the **n − 1 non-maximum elements must lose at least one comparison**, and one comparison makes only one loser ⇒ **n − 1**, which a linear scan achieves.

Max and min together: the pairwise method: Doing them separately costs `(n - 1) + (n - 1) = 2n - 2`. Better: **compare the elements in pairs** (n/2 comparisons); the max is among the **winners** (n/2 − 1 more), the min among the **losers** (n/2 − 1 more). Total `3n/2 - 2` for even n; for odd n start max = min = first element and pair the rest ⇒ in general **⌈3n/2⌉ − 2**, and an adversary argument proves no algorithm does better.



the pairwise method on 8 numbers: each pair comparison sends one number up and one down

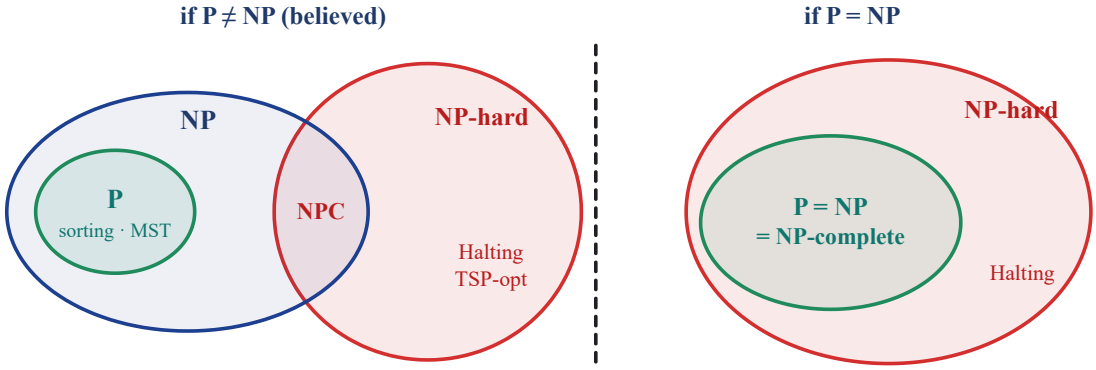
Problem (comparisons, n items)	Lower bound	Optimal algorithm
Maximum (or minimum)	n − 1	linear scan
Maximum and minimum	**⌈3n/2⌉ − 2**	pairwise method
Second largest	**n + ⌈log₂ n⌉ − 2**	tournament + max's victims
Search a sorted array	⌈log₂(n + 1)⌉	binary search
Search an unsorted array	n	linear search
Merge two sorted lists of n	2n − 1	standard merge
Comparison sorting	⌈log₂ n!⌉	merge sort, heapsort
Element uniqueness	Ω(n log n)	sort, then scan
Connectivity (adj. lists)	Ω(V + E)	BFS / DFS

Worked ✍ n = 100: Max + min: `⌈300/2⌉ - 2 = 148` (50 pairs + 49 + 49), not 198. Second largest: `100 + ⌈log₂ 100⌉ - 2 = 100 + 7 - 2 = 105`. With n = 8: **7** games find the champion, who beat **3** players; the best of those 3 needs **2** more ⇒ 9 = 8 + 3 − 2.

Exam trap: **2n − 2** (or 2n − 3) for max + min is the naive count, not the bound. Do not confuse **⌈3n/2⌉ − 2** (max + min) with **n + ⌈log₂ n⌉ − 2** (second largest). "Minimum comparisons in the worst case" means the **lower bound**, not the lucky best case.

7 P, NP, co-NP, NP-Hard and NP-Complete

Class	Definition (yes/no decision problems)	Examples
P	solved deterministically in **polynomial time $O(n^k)$**	sorting, MST, shortest path, 2-SAT, LP, primality
NP	a "yes" has a **certificate verifiable in polynomial time** (= nondeterministic poly time)	all of P; SAT, Ham. cycle, subset sum, factoring
co-NP	complements: "no" answers have short certificates	TAUTOLOGY, UNSAT
NP-hard	**every** NP problem ≤p it; need not be in NP or decidable	Halting problem, TSP-optimisation
NP-complete	**in NP and NP-hard** — the hardest of NP	SAT, 3-SAT, clique, vertex cover, TSP-decision



NP-complete is the overlap of NP and NP-hard; NP-hard reaches outside NP in both worlds

Common misconception ✍️: A nested picture with **P inside NP inside NP-complete inside NP-hard** is wrong. NP-complete is **not** a superset of NP: it is the part of NP that is also NP-hard, and NP-hard spills **outside** NP. **P ⊆ NP** is proven (a solver is its own verifier); whether **P = NP** is open.

Decision vs optimisation: **TSP-decision**: "is there a tour of cost ≤ k?" — in NP, NP-complete. **TSP-optimisation**: "find the cheapest tour" — NP-hard, not in NP (not a yes/no problem). Same for knapsack, clique, vertex cover. Solving the optimisation version solves the decision version, so it is at least as hard.

What a certificate looks like ✍️: **SAT:** a truth assignment — plug it in, O(formula length). **Hamiltonian cycle:** an ordering of the n vertices — check n edges exist and nobody repeats. **Subset sum:** the subset — add it up. Finding is hard; **checking** is easy.

Memory trick: "**P: solve fast. NP: check fast. NP-hard: at least as hard as all of NP. NP-complete: hard and checkable.**" And **NP = Nondeterministic Polynomial**, never "Non-Polynomial" — every P problem is in NP.

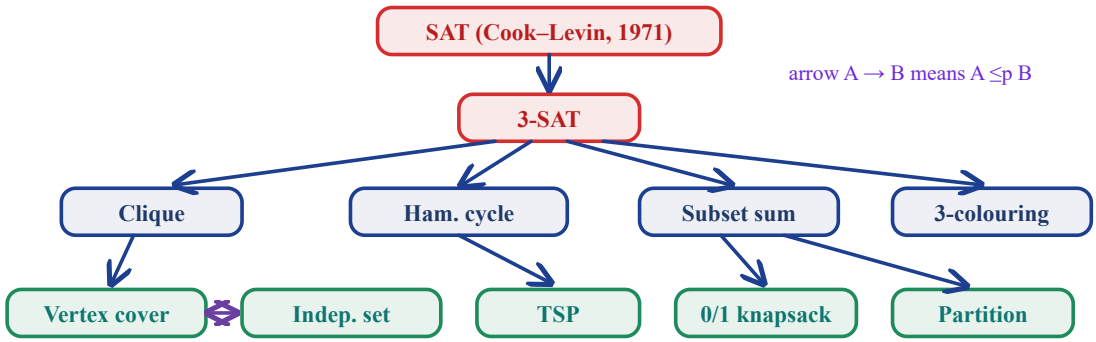
Where famous problems sit: **P ⊆ NP ∩ co-NP. Primality** is in P (AKS, 2002) — testing is easy, while **factoring** is in NP ∩ co-NP but not known to be in P or NP-complete. **Graph isomorphism**: in NP, not known NP-complete (quasi-polynomial, 2015). If P ≠ NP, Ladner's theorem says such in-between problems must exist.

8 Reductions and the Cook–Levin Theorem

Reduction A ≤p B: A polynomial-time computable function f with `x ∈ A ⇔ f(x) ∈ B` for every instance x. Read it as "**A is no harder than B**": a fast algorithm for B gives one for A. Two consequences: **B ∈ P ⇒ A ∈ P**, and **A NP-hard ⇒ B NP-hard**. Reductions compose: A ≤p B and B ≤p C ⇒ A ≤p C.

Cook–Levin theorem: **SAT is NP-complete** — Stephen **Cook (1971)**, Leonid **Levin** independently (1973). It was the **first** problem proved NP-complete, directly from the definition: any polynomial-time nondeterministic machine's run is encoded as a Boolean formula. **Karp (1972)** then reduced SAT to **21** classic problems.

Proving X NP-complete ✍ 4 steps: **1.** X ∈ NP: give a certificate and a polynomial-time verifier. **2.** Pick a **known** NP-complete A. **3.** Build f from A-instances to X-instances in polynomial time. **4.** Prove `x ∈ A ⇔ f(x) ∈ X` (both directions). Steps 2–4 alone prove X **NP-hard**.



the standard textbook reduction tree: hardness flows down the arrows

Worked ✍ Clique ≤p Vertex cover: S is an **independent set** of G ⇔ V − S is a **vertex cover** of G, and S is a **clique** of G ⇔ S is independent in the complement $\bar{G}$. So `(G, k) ↦ (Ḡ, |V| - k)`. Example: V = {1,2,3,4}, edges 12, 13, 23, 34; clique {1,2,3} (k = 3). $\bar{G}$ has edges 14, 24 — covered by **{4}**, size 4 − 3 = 1 ✓.

Worked ✍ Ham. cycle ≤p TSP: From G with n vertices build a complete graph: weight **1** on G's edges, **2** on non-edges. G has a Hamiltonian cycle ⇔ there is a tour of cost **≤ n**. Building the n^2 weights is polynomial, so TSP-decision is NP-hard; a tour is checkable, so it is NP-complete.

Exam trap ✍ the direction: To show X is hard, reduce a **known** NP-complete problem **to X**: **A ≤p X**. Reducing X to SAT (X ≤p SAT) only shows X is **no harder** than SAT (so X ∈ NP) — nothing about hardness. "Hard problem → new problem", never the reverse.

2-SAT vs 3-SAT: Clauses of **two** literals: build the **implication graph** (a ∨ b gives a′ → b and b′ → a); satisfiable ⇔ no variable shares a strongly connected component with its negation ⇒ **2-SAT ∈ P, linear time**. With **three** literals per clause the problem is NP-complete. One extra literal crosses the line.

9 Easy/Hard Twins, P vs NP, Coping

In P (polynomial)	Looks similar, but NP-complete	The difference
2-SAT, Horn-SAT	**3-SAT**, SAT	3 literals per clause
Euler circuit (each **edge** once)	**Hamiltonian cycle** (each **vertex**)	edges vs vertices
Shortest simple path	**Longest** simple path	min vs max
Minimum spanning tree	TSP; Steiner tree	tree vs tour
2-colouring (bipartite)	**3-colouring**	2 vs 3 colours
Edge cover, matching	Vertex cover, 3-D matching	vertices, 3 sets
Linear programming	0-1 integer programming	real vs integer
Fractional knapsack	0/1 knapsack (decision)	split vs whole

P vs NP: If **any one** NP-complete problem has a polynomial algorithm, then **P = NP** and every NP problem does. If P ≠ NP, no NP-complete problem is in P (and, by Ladner's theorem, some NP problems are neither in P nor NP-complete — factoring and graph isomorphism are suspects). The question is one of the seven **Clay Millennium Prize Problems** (2000, US$1 million); most researchers believe P ≠ NP.

The IT link: Public-key cryptography relies on problems believed hard (factoring for **RSA**, discrete log for Diffie–Hellman). They lie in NP, so a proof of P = NP with a practical algorithm would break them. Timetabling, vehicle routing, register allocation (graph colouring) and chip layout are NP-hard problems solved daily by the methods below.

Coping strategy	You give up	Example
Smart exponential	speed on large n	backtracking, branch-and-bound; Held–Karp TSP **$O(n^2 2^n)$** vs O(n!)
Pseudo-polynomial DP	big numbers	0/1 knapsack **O(nW)**; subset sum O(nS)
Approximation	optimality (ratio proven)	vertex cover 2×; metric TSP 2× (MST), Christofides 1.5×
Heuristics	any guarantee	greedy, simulated annealing, genetic, tabu search
Special cases	generality	2-SAT; vertex cover on trees; small k: $O(2^k \cdot n)$

Exam trap ✍ pseudo-polynomial: Knapsack's DP is **O(nW)**, yet knapsack is NP-complete — no contradiction. W is written in **$\log_2$ W bits**, so nW is **exponential in the input length**. Doubling the bits of W squares W. Such problems are called **weakly** NP-complete.

Memory trick: "**Euler edges, Hamilton heads.**" Every edge once is easy (degree check), every vertex once (the "heads") is NP-complete. Same pattern: **shortest easy, longest hard; 2 easy, 3 hard**. Ratio-guaranteed methods are taught in the approximation-algorithms lesson.

Night-Before-the-Exam Sheet

Fact sheet:

Horner: n mult + n add, optimal

Horner = synthetic division, last = $p(x_0)$

Fast power: $\leq 2\lfloor \log_2 n \rfloor$ mult, $O(\log n)$

x^{13}: 5 multiplications

Karatsuba: $3T(n/2)+O(n) =$ **$n^{1.585}$**

$ad+bc = (a+b)(c+d) - ac - bd$

Strassen: 7 products, $n^{\log_2 7} \approx n^{2.81}$

FFT poly multiply: $O(n \log n)$

$A + BC = (A+B)(A+C)$

$AB + AB' = A$, $A + A'B = A + B$

O upper · Ω lower · Θ tight (functions)

$\Omega \neq$ best case

Sort: $h \geq \log_2 n! = \Omega(n \log n)$

3 keys: 6 leaves, 3 comparisons

Max: $n - 1$ · max+min: $\lceil 3n/2 \rceil - 2$

2nd largest: $n + \lceil \log_2 n \rceil - 2$

Sorted search: $\lceil \log_2(n+1) \rceil$

Merge $n + n$: $2n - 1$

NP: verify in poly time

NPC = NP ∩ NP-hard

Cook 1971 · Levin 1973 · Karp 1972 (21)

$A \leq_p X$, A known NPC ⇒ X NP-hard

2-SAT ∈ P · 3-SAT NPC

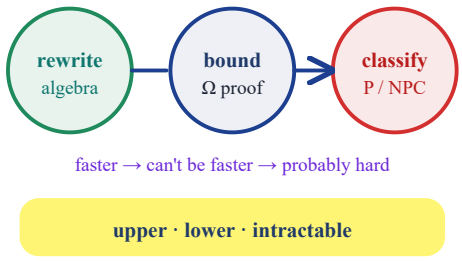
Knapsack $O(nW)$: pseudo-polynomial

Top 12 one-liners for MCQs

1. Presorting = instance simplification
2. Binary → decimal is Horner with x = 2
3. Strassen exponent 2.81, not 2.37
4. Matrix product: trivial bound $\Omega(n^2)$
5. Counting / radix sort beat n log n
6. n = 100: max + min = 148
7. NP = Nondeterministic Polynomial
8. P ⊆ NP proven; P = NP open
9. Halting: NP-hard, not NP-complete
10. TSP-opt NP-hard; TSP-decision NPC
11. Euler circuit P · Hamiltonian NPC
12. Clay Millennium Prize: US$1 million

Top traps: Horner "n + 1" or "2n" multiplications · Ω read as best case · comparison bound applied to counting sort · max + min "2n − 2" · nested P ⊂ NP ⊂ NPC ⊂ NP-hard picture · reducing X to SAT to "prove" X hard · O(nW) taken as polynomial · NP read as "non-polynomial".

All tricks together: Horner = nested brackets = n and n · save one multiplication, lower the exponent · P solve fast, NP check fast, NPC hard and checkable · Euler edges, Hamilton heads · shortest easy, longest hard; 2 easy, 3 hard · hard problem → new problem.



the lesson in three moves

✓ Practice MCQs (exam pattern, 10 marks)

1. Horner's rule evaluates a polynomial of degree n using how many multiplications?
(a) $n(n+1)/2$ (b) $2n-1$
(c) n (d) $\log_2 n$

2. Karatsuba's algorithm multiplies two n-digit integers in:
(a) $\Theta(n^2)$ (b) $\Theta(n^{1.585})$
(c) $\Theta(n \log n)$ (d) $\Theta(n^{2.81})$

3. Strassen's method multiplies 2×2 block matrices with how many block multiplications?
(a) 7 (b) 8
(c) 6 (d) 18

4. Worst-case lower bound for any comparison-based sort of n elements:
(a) $\Omega(n)$ (b) $\Omega(\log n)$
(c) $\Omega(n^2)$ (d) $\Omega(n \log n)$

5. Minimum worst-case comparisons to find both max and min of 100 distinct numbers:
(a) 198 (b) 150
(c) 148 (d) 99

6. A decision tree that sorts 4 distinct keys must have height at least:
(a) 4 (b) 5
(c) 6 (d) 24

7. Which of these problems is NP-complete?
(a) 2-SAT (b) 3-SAT
(c) Minimum spanning tree (d) Shortest path, non-negative weights

8. To prove a new problem X is NP-hard, you must:
(a) Reduce X to SAT (b) Show a certificate for X is checkable
(c) Give an exponential algorithm for X (d) Reduce a known NP-complete problem to X

9. The Halting problem is:
(a) NP-complete (b) In P
(c) NP-hard but not in NP (d) In NP but not NP-hard

10. The O(nW) dynamic program for 0/1 knapsack is:
(a) Pseudo-polynomial (b) Polynomial in the input length
(c) A proof that P = NP (d) An approximation algorithm

Answer key → 1-c · 2-b · 3-a · 4-d · 5-c · 6-b · 7-b · 8-d · 9-c · 10-a

Written question: (a) Prove that every comparison sort needs $\Omega(n \log n)$ comparisons in the worst case, using the decision tree for three keys. (b) Show how to find both the maximum and the minimum of n numbers in $\lceil 3n/2 \rceil - 2$ comparisons and evaluate it for $n = 100$. (c) Define P, NP, NP-hard and NP-complete, draw their relationship, and list the steps to prove a problem NP-complete. 10 marks

1. **(a) Decision tree (4)** — each comparison is a node, each of the n! orderings needs its own leaf; height $h \Rightarrow \leq 2^h$ leaves $\Rightarrow h \geq \log_2 n! \geq (n/2)\log_2(n/2) = \Omega(n \log n)$. For 3 keys: 6 leaves, worst case $\lceil \log_2 6 \rceil = 3$. 4
2. **(b) Pairwise method (3)** — compare in pairs (n/2), max of winners $(n/2 - 1)$, min of losers $(n/2 - 1) \Rightarrow 3n/2 - 2$; $n = 100$: $50 + 49 + 49 = 148$ (vs 198 separately). 3
3. **(c) Classes (3)** — P: solvable in poly time; NP: verifiable in poly time; NP-hard: every NP problem $\leq_p$ it; NPC = NP ∩ NP-hard; draw $P \subseteq NP$, NPC as the overlap, NP-hard outside. Steps: $X \in NP$, pick known NPC A, build f in poly time, prove $x \in A \Leftrightarrow f(x) \in X$. 3

Before the exam ✓

- ✓ Can run Horner / synthetic division
- ✓ Can state Karatsuba and Strassen exponents
- ✓ Can draw the 3-key decision tree
- ✓ Can derive $\lceil 3n/2 \rceil - 2$
- ✓ Can draw the correct P / NP / NPC picture
- ✓ Know which way a reduction goes